PRODUCT & TECHNICAL OVERVIEW

KnightVision

AI-powered chess game analysis — Stockfish accuracy scores, LLM coaching commentary, and a real-time review board in the browser.

♟ Chess Analysis Platform

⚡ Real-time WebSocket Streaming

🤖 Multi-LLM Provider

📦 Docker Single-Command

1 What is KnightVision?

KnightVision is a full-stack chess game analysis tool that combines the world-class Stockfish chess engine with AI language models to deliver move-by-move coaching commentary. A player pastes a PGN game, clicks **Analyse**, and within seconds sees every move colour-coded by quality — Brilliant, Best, Inaccuracy, Blunder — alongside a coaching explanation, an evaluation bar, and the top alternative moves the engine recommended.

The goal is to make post-game analysis as accessible as a chess.com or Lichess review, but with the flexibility to run entirely locally (no cloud required) or swap in a free cloud LLM (Groq) when needed.

> **Core value proposition:** Upload any PGN → get a fully annotated game with accuracy scores, move-quality badges, and plain-English coaching comments — in under 60 seconds — with zero setup beyond `docker compose up`.

2 Key Features

♟ Stockfish Move Analysis

Every position evaluated at depth 18. Centipawn loss computed per move from the moving player's perspective. Top 3 alternatives returned for each position.

🤖 AI Coaching Commentary

LLM generates a 1–2 sentence plain-English explanation for every move. Supports Ollama (local), Groq, and HuggingFace. Falls back to templates if LLM times out.

🎨 Colour-Coded Move List

All 7 move categories displayed with distinct colours and badges. Move list is scrollable with auto-scroll to the current move. WCAG 2.1 AA contrast verified.

📊 Accuracy Scores

White and Black accuracy percentages calculated across the full game. Displayed in the game header with colour-coded indicators (green ≥80%, amber 60–79%, red <60%).

⚡ Real-Time Streaming

Results stream via WebSocket as each move is analysed — no waiting for the whole game. Late-joining clients receive buffered replay instantly without animation delays.

📱 Responsive & Accessible

Fully responsive layout — desktop two-column, mobile bottom-sheet with swipe navigation. Keyboard nav (arrow keys, Space, Home, End). WCAG 2.1 AA compliant.

🔄 Runtime LLM Switching

Switch between Ollama, Groq, and HuggingFace at runtime via the UI dropdown — no server restart required. Provider health displayed live.

🐳 Single-Command Docker

Three services (backend, frontend, Ollama) start with one command. Ollama model is auto-pulled and cached in a Docker volume on first run.

3 Move Classification System

Every move is classified into one of **7 quality tiers** based on the centipawn loss — the difference between the engine's evaluation before and after the move, from the moving player's perspective.

Badge	cp_loss threshold	Meaning
Brilliant	`= 0` + sacrifice	Zero loss + piece sacrifice detected
Great	`≤ 5`	Near-perfect move
Best	`≤ 10`	Engine's top recommendation
Good	`≤ 20`	Solid, slightly sub-optimal
Inaccuracy	`≤ 50`	Noticeable error
Mistake	`≤ 150`	Significant error
Blunder	`> 150`	Game-changing error

Brilliant detection: A move must have cp_loss = 0 AND involve a piece sacrifice (giving up material that isn't immediately recoverable). This prevents trivially recapturing moves from being labelled Brilliant.

Configurable: All thresholds are adjustable in `config.yaml` under `classification.thresholds` — no code change needed.

4 LLM Provider System

KnightVision's LLM layer is built around a **pluggable provider registry** with automatic fallback. All providers implement the same interface (`generate(prompt) → str` + `check_health() → bool`), making them interchangeable at runtime.

Provider	Model	Key Required	Best For
Ollama (default)	`llama3.1:8b`	None — runs locally	Privacy, offline use, no API costs
Groq	`llama3-8b-8192`	`GROQ_API_KEY` (free tier)	Fastest inference, hosted deployments
HuggingFace	`Mistral-7B-Instruct-v0.2`	`HUGGINGFACE_API_KEY` (free)	Alternative cloud option
Fallback	Template engine	None — always available	Guaranteed comment if all LLMs fail

Fallback Chain

Every comment request goes through `generate_with_fallback()` : it tries the active provider, and if that fails (network error, timeout, no API key), automatically tries the next registered provider, finally falling back to a template comment. Analysis **never fails** due to an LLM being unavailable.

10-Second Timeout

Each LLM call has a 10-second timeout. If the model doesn't respond in time, the fallback kicks in immediately — preventing slow LLM responses from blocking the entire analysis pipeline.

5 Architecture Overview

KnightVision is a standard full-stack web application with a Python/FastAPI backend, a React/TypeScript frontend, and two sidecar services (Stockfish subprocess, Ollama container).

```
Browser (React + TypeScript + Zustand)
|
| POST /api/analysis/upload ──→ 202 Accepted { game_id }
| WebSocket /ws/analysis/{id} ←── move_result stream + analysis_complete
| GET /api/llm/status ──→ provider health map
|
▼
Nginx (port 3000 → serves Vite build + proxies /api and /ws)
|
▼
FastAPI Backend (Python 3.12, port 8000)
|
├── Analysis Orchestrator (per-game background task)
| |
| ├── PGN Parser (python-chess) ──→ move list + metadata
| |
| ├── Stockfish Service (asyncio.to_thread)
| | └── UCI subprocess at depth 18, multipv 3
| | returns: cp_score, best_move, top_candidates
| |
| ├── Move Classifier ──→ category (7 tiers via cp_loss)
| |
| ├── LLM Provider Registry
| | ├── OllamaProvider → http://ollama:11434
| | ├── GroqProvider → api.groq.com
| | ├── HuggingFaceProvider→ api-inference.huggingface.co
| | └── FallbackProvider → template comment (always available)
| |
| └── Opening Service ──→ ECO code + opening name
|
├── WebSocket Handler ──→ streams results to browser
| ├── live stream: buffered=false → triggers board animation
| ├── late-join: buffered=true → instant replay, no animation
| └── heartbeat every 10s
|
└── In-Memory Game Store (game_id → status + moves + metadata)

Ollama sidecar (port 11434, Docker volume: ollama_data)
└── llama3.1:8b — local LLM inference, no cloud required
```

6 Technology Stack

Backend

Python 3.12 · FastAPI · python-chess · Stockfish (UCI) · asyncio · Pydantic v2 · WebSockets · httpx · pytest

Frontend

React 18 · TypeScript · Zustand 4 · Vite · Tailwind CSS 3 · react-chessboard · chess.js · Vitest · axe-core

Infrastructure

Docker Compose · Nginx · Ollama · Railway

LLM Providers

Ollama (local) · Groq API · HuggingFace

7 Request Data Flow

A full game analysis follows this sequence from browser to results:

#	Step	Component	Output
1	User pastes PGN and clicks Analyse	UploadZone (React)	POST `/api/analysis/upload`
2	PGN validated & game_id assigned	FastAPI upload endpoint	`202 { game_id }`
3	Browser opens WebSocket connection	`useAnalysis` hook	WS `/ws/analysis/{game_id}`
4	PGN parsed into move list + metadata	python-chess parser	List of SAN moves + headers
5	Opening detected from ECO database	Opening Service	ECO code + opening name
6	Per move: Stockfish evaluates position	StockfishService (thread)	cp_score, best_move, candidates
7	cp_loss computed & move classified	MoveClassifier	Category enum (7 tiers)
8	LLM generates coaching comment	ProviderRegistry	Comment string or fallback template
9	move_result streamed to browser	WebSocket handler	`{ type: "move_result", ... }`
10	Store updated, board & list re-render	Zustand store	React components update live
11	analysis_complete with accuracy scores	WebSocket handler	`{ white_accuracy, black_accuracy }`

8 Frontend State & Components

Zustand Store — analysisStore

Single source of truth for all UI state. All components subscribe to slices of this store.

Key React Components

State field	Type	Purpose
`moves`	`MoveResult[]`	All analysed moves
`analysisStatus`	6-state enum	idle → uploading → analysing → complete / error / stalled
`gameId`	`string`	Current game UUID
`meta`	`GameMeta`	Players, date, result, opening
`activeProvider`	`LLMProvider`	Selected LLM provider
`providerHealth`	`Record`	Health bool per provider
`whiteAccuracy`	`number`	White accuracy %
`blackAccuracy`	`number`	Black accuracy %

Component	Role
`UploadZone`	PGN input, drag-drop, validation
`ReviewLayout`	Root layout — desktop/mobile switch
`Board`	Chessboard + best-move arrow + swipe
`EvalBar`	Vertical evaluation bar
`MoveList`	Scrollable colour-coded move list
`MoveDetailCard`	Badge, eval delta, comment, candidates
`PlaybackControls`	⏮ ◀ ⏸/▶ ▶ ⏭ + move counter
`GameHeader`	Players, Elo, date, opening, accuracy
`BottomSheet`	Mobile slide-up move detail panel

9 Streaming & Resilience

📡

WebSocket Streaming

Results are pushed to the browser as each move is processed — the board updates in real time. A 10-second heartbeat keeps the connection alive through proxies and firewalls.

⏪

Buffered Replay

A client that reconnects mid-analysis immediately receives all prior moves with `buffered: true` — rendered instantly without animation — then seamlessly transitions to live streaming.

⏱️

Stall Detection

The frontend monitors time since last WebSocket message. If no message arrives in 30 seconds while analysis is active, status transitions to *stalled* and a warning banner is shown.

🛡️

Per-Move Error Isolation

If a single move fails (Stockfish error, LLM timeout), the error is isolated to that move — a fallback "Good" category and template comment are used — and analysis continues for all remaining moves.

10 Accessibility (WCAG 2.1 AA)

Colour Contrast

All 7 badge colours verified at ≥ 4.5:1 contrast ratio against the dark background:

Brilliant	6.93:1	#1baaa6 / dark text
Great	7.00:1	#5ca0d3 / dark text
Best	8.37:1	#6dba6a / dark text
Good	9.04:1	#96bc4b / dark text
Inaccuracy	11.80:1	#f0c15c / dark text
Mistake	7.34:1	#e8834e / dark text
Blunder	5.20:1	#ca3431 / white text

Keyboard & Screen Reader

- ← → Space Home End for playback navigation
- `:focus-visible` ring — 2px #5ca0d3, offset 2px
- Move list items: `aria-label="Move N: SAN, category"`
- Move list items: `aria-pressed` for toggle state
- All icon-only buttons have `aria-label`
- Board: piece icons are primary signal (not colour alone)
- axe-core gate test: 0 critical/serious violations

11 Deployment

Local Docker (Default)

Cloud (Railway + Groq)

```
git clone <repo> && cd knightvision
cp .env.example .env
docker compose up --build
```

Three services start: **backend** (port 8000), **frontend** (port 3000), **Ollama** (port 11434). The `llama3.1:8b` model (~4 GB) is pulled automatically on first run and cached in a Docker volume.

```
railway init
railway up
# set in Railway dashboard:
# GROQ_API_KEY=...
# LLM_DEFAULT_PROVIDER=groq
```

Backend deploys from `backend/Dockerfile`. Frontend deploys to Vercel/Netlify with `VITE_API_BASE_URL` pointing to the Railway backend URL.

Prod Compose Override (no Ollama)

```
docker compose -f docker-compose.yml -f docker-compose.prod.yml up --build
```

Removes the Ollama sidecar and switches the default provider to Groq. Useful for low-memory machines or when a cloud LLM is preferred locally.

12 Configuration Reference

config.yaml key	Default	Description
`stockfish.depth`	`18`	Analysis depth (10–22). Higher = slower but more accurate.
`stockfish.threads`	`2`	CPU threads allocated to Stockfish subprocess
`stockfish.multipv`	`3`	Top candidate moves returned per position
`stockfish.path`	`null`	Auto-detect binary; set to explicit path if needed
`llm.default_provider`	`"ollama"`	Active provider at startup. Override with `LLM_DEFAULT_PROVIDER` env var
`llm.timeout_seconds`	`10`	LLM response timeout before fallback template
`llm.ollama.model`	`llama3.1:8b`	Ollama model name
`llm.groq.model`	`llama3-8b-8192`	Groq model name
`server.cors_origins`	`[localhost:5173, ...]`	Allowed CORS origins. Override with `ALLOWED_ORIGINS` env var (comma-separated)
`accuracy.max_cp_scale`	`300`	CP loss at which accuracy = 0%

13 Performance & Quality Targets

Requirement	Target
Full game analysis (54 moves, depth 18)	< 60 seconds
WebSocket first result after upload	< 3 seconds
Board render / move animation	< 100 ms
LLM timeout before fallback	10 seconds
Mobile board minimum size	300 × 300 px
Browser support	Chrome 90+, Firefox 90+, Safari 15+, Mobile Safari
Accessibility	WCAG 2.1 AA (axe-core gate: 0 critical/serious violations)
Frontend tests	289 tests, 0 regressions
Backend tests	269 tests, 0 regressions